

1. Retrieve the title and price of all courses in the 'Data Science' category that cost less than 2000. Order the results from the cheapest to the most expensive.

Query :-- select title,price from Courses where category='Data Science' and price<2000
order by price asc

The screenshot shows a PostgreSQL query editor interface. The left sidebar displays a tree view of the database schema, including 'public' and 'Foreign Data'. The main query editor displays the following SQL query:

```
-----1-----
49 select title,price from Courses
50 where category='Data Science' and price<2000
51 order by price asc
52
-----2-----
53
54 select category,count(*) as total_courses
55 from Courses group by category
56
-----3-----
57
58 select C.category ,count(enrollment_id) as Total_Enrollments
59 from Courses as C inner join Enrollments as E
60 on C.course_id=E.course_id group by C.category having count(enrollment_id)
61
-----4-----
62
63
```

The 'Data Output' tab shows the results of the first query:

	title	price
1	NLP with Transformers - Adva...	999
2	Angular Deep Dive - Advanced	999
3	SQL Basics - Beginner	1499
4	Data Visualization - Intermedi...	1499
5	Hadoop & Spark - Beginner	1499

2. Determine the number of courses offered in each distinct category.

Query:--- select category,count(*) as total_courses from Courses group by category

The screenshot shows a PostgreSQL query editor interface. The left sidebar displays a tree view of the database schema, including 'public' and 'Foreign Data'. The main query editor displays the following SQL query:

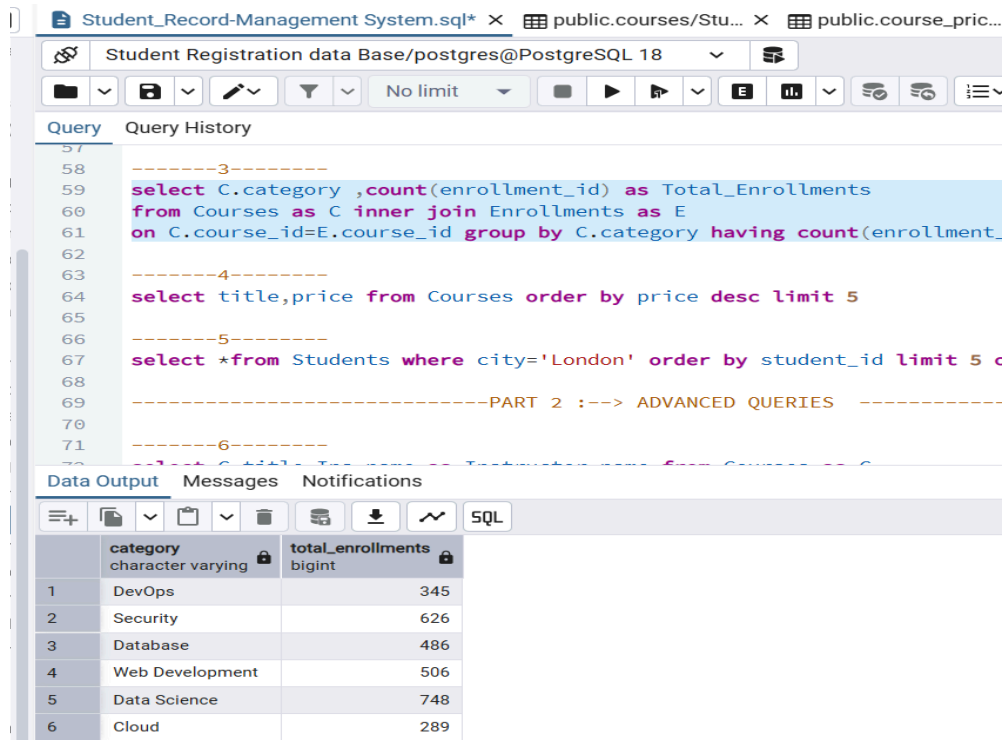
```
-----2-----
54 select category,count(*) as total_courses
55 from Courses group by category
56
-----3-----
57
58 select C.category ,count(enrollment_id) as
59 from Courses as C inner join Enrollments a
60 on C.course_id=E.course_id group by C.cate
61
-----4-----
62
63 select title,price from Courses order by p
64
-----5-----
65
66 select *from Students where city='London' ,
67
68
```

The 'Data Output' tab shows the results of the second query:

	category	total_courses
1	DevOps	5
2	Security	8
3	Database	6
4	Web Development	7
5	Data Science	10
6	Cloud	4

3. Identify the categories where the total number of enrollments is greater than 150. Display the category name and the total enrollment count.

Query :--- select C.category ,count(enrollment_id) as Total_Enrollments
from Courses as C inner join Enrollments as E on C.course_id=E.course_id group
by C.category having count(enrollment_id)>150



Student Record-Management System.sql* x public.courses/Stu... x public.course_pric...

Student Registration data Base/postgres@PostgreSQL 18

Query History

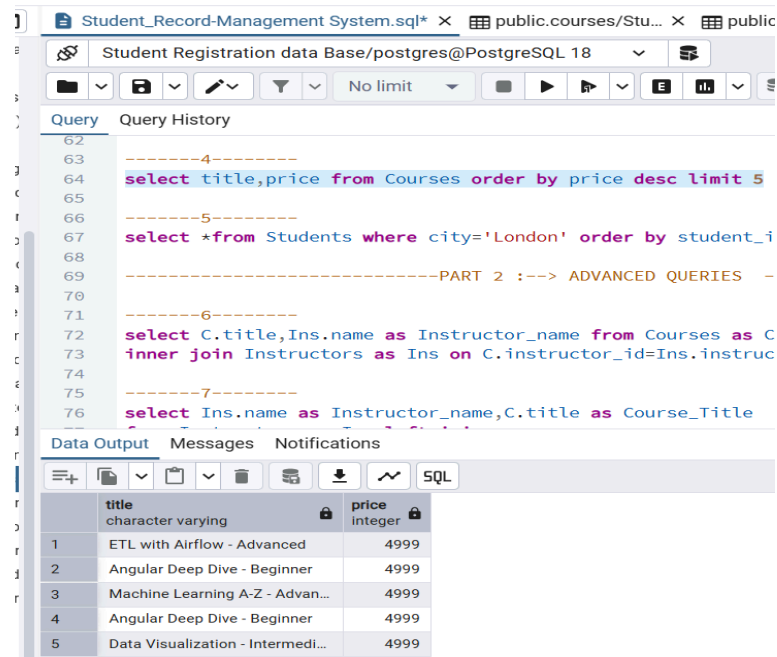
```
57  
58 -----3-----  
59 select C.category ,count(enrollment_id) as Total_Enrollments  
60 from Courses as C inner join Enrollments as E  
61 on C.course_id=E.course_id group by C.category having count(enrollment  
62  
63 -----4-----  
64 select title,price from Courses order by price desc limit 5  
65  
66 -----5-----  
67 select *from Students where city='London' order by student_id limit 5 c  
68  
69 -----PART 2 :--> ADVANCED QUERIES -----  
70  
71 -----6-----  
72 select C.title,Ins.name as Instructor_name from Courses as C  
73 inner join Instructors as Ins on C.instructor_id=Ins.instruc
```

Data Output Messages Notifications

	category character varying	total_enrollments bigint
1	DevOps	345
2	Security	626
3	Database	486
4	Web Development	506
5	Data Science	748
6	Cloud	289

4. Find the top 5 most expensive courses. Display the course title and its price.

Query:--- select title,price from Courses order by price desc limit 5



Student Record-Management System.sql* x public.courses/Stu... x public

Student Registration data Base/postgres@PostgreSQL 18

Query History

```
62  
63 -----4-----  
64 select title,price from Courses order by price desc limit 5  
65  
66 -----5-----  
67 select *from Students where city='London' order by student_i  
68  
69 -----PART 2 :--> ADVANCED QUERIES -  
70  
71 -----6-----  
72 select C.title,Ins.name as Instructor_name from Courses as C  
73 inner join Instructors as Ins on C.instructor_id=Ins.instruc  
74  
75 -----7-----  
76 select Ins.name as Instructor_name,C.title as Course_Title
```

Data Output Messages Notifications

	title character varying	price integer
1	ETL with Airflow - Advanced	4999
2	Angular Deep Dive - Beginner	4999
3	Machine Learning A-Z - Advan...	4999
4	Angular Deep Dive - Beginner	4999
5	Data Visualization - Intermedi...	4999

5. List the 6th through 10th students (by ascending student_id) who live in 'London'.

Query:-- select *from Students where city='London' order by student_id limit 5 offset 5

The screenshot shows a PostgreSQL query editor with the following SQL query highlighted:

```
select *from Students where city='London' order by student_id limit 5 offset 5
```

Below the query editor, the 'Data Output' tab is active, displaying a table with 5 rows of student data:

student_id [PK] integer	name character varying	email character varying	city character varying	
1	59	Mia Thompson	mia.thompson1036@studentmail.com	London
2	86	Sophia Anderson	sophia.anderson9038@studentmail.c...	London
3	88	Casey Robinson	casey.robinson2188@studentmail.com	London
4	103	Jamie Walker	jamie.walker4449@studentmail.com	London
5	153	Casey Clark	casey.clark3077@studentmail.com	London

6. List the title of every course along with the name of the instructor who teaches it.

Query:--select C.title,Ins.name as Instructor_name from Courses as C
inner join Instructors as Ins on C.instructor_id=Ins.instructor_id

The screenshot shows a PostgreSQL query editor with the following SQL query highlighted:

```
select C.title,Ins.name as Instructor_name from Courses as C  
inner join Instructors as Ins on C.instructor_id=Ins.instructor_id
```

Below the query editor, the 'Data Output' tab is active, displaying a table with 8 rows of course and instructor data:

title character varying	Instructor_name character varying
1 Cybersecurity 101 - Intermediate	Sophia Garcia
2 SQL Basics - Beginner	Sophia Garcia
3 ETL with Airflow - Advanced	Ava Taylor
4 Data Visualization - Intermediate	Sophia Garcia
5 Angular Deep Dive - Advanced	Ava Taylor
6 Docker & Kubernetes - Beginner	Quinn White
7 React for Beginners - Intermediate	Riley White
8 Introduction to Databases - Beginner	Robin Garcia

- Display the name of every instructor. If an instructor is teaching any courses, also list the title of their courses. If an instructor is not teaching any courses, their name should still appear with a blank or NULL course title.

Query :--select Ins.name as Instructor_name,C.title as Course_Title
from Instructors as Ins left join Courses as C on Ins.instructor_id=C.instructor_id

The screenshot shows a PostgreSQL IDE window titled 'Student_Record-Management System.sql*'. The query editor contains the following SQL query:

```
--7--
select Ins.name as Instructor_name,C.title as Course_Title
from Instructors as Ins left join
Courses as C on Ins.instructor_id=C.instructor_id
```

The 'Data Output' tab shows the results of the query:

	instructor_name	course_title
1	Sophia Garcia	Cybersecurity 101 - Intermediate
2	Sophia Garcia	SQL Basics - Beginner
3	Ava Taylor	ETL with Airflow - Advanced
4	Sophia Garcia	Data Visualization - Intermediate
5	Ava Taylor	Angular Deep Dive - Advanced
6	Quinn White	Docker & Kubernetes - Beginner

- Find the student name, course title, and the instructor name for all enrollments made by students living in the city 'New York'.

Query:--select Stu.name as Student_Name ,C.title as Course_Title,Ins.name as
Instructors_Name from Students as Stu inner join Enrollments as E on
Stu.student_id=E.student_id inner join Courses as C on E.course_id=
C.course_id inner join Instructors as Ins on C.instructor_id=Ins.instructor_id
where Stu.city='New York'

The screenshot shows a PostgreSQL IDE window titled 'Student_Record-Management System.sql*'. The query editor contains the following SQL query:

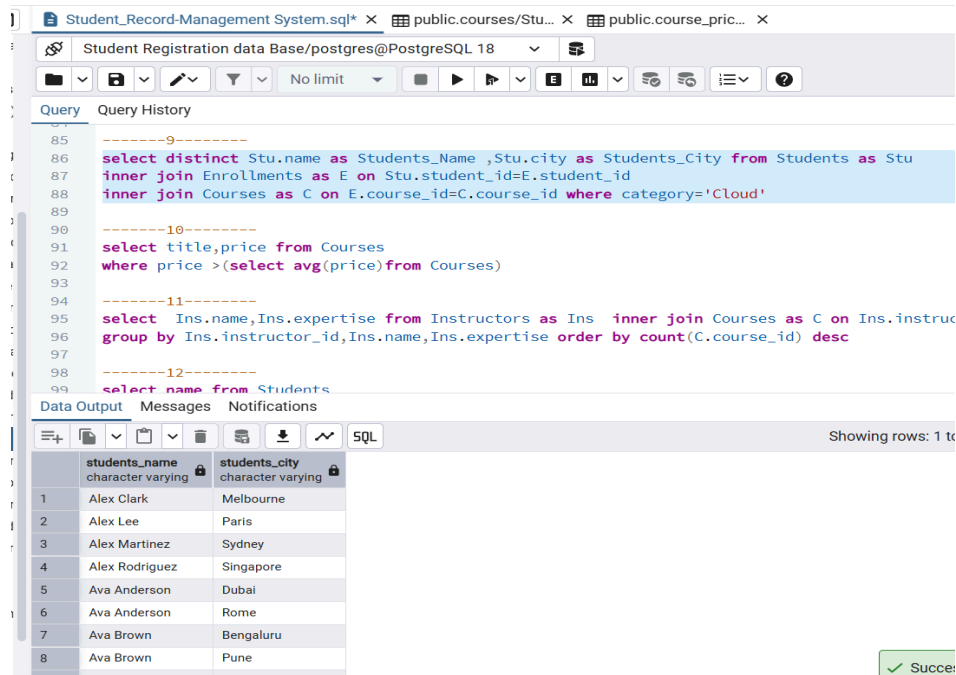
```
--8--
select Stu.name as Student_Name ,C.title as Course_Title,Ins.name as Instructors_Name from Students as Stu
inner join Enrollments as E on Stu.student_id=E.student_id inner join Courses as C on E.course_id=C.course_id
inner join Instructors as Ins on C.instructor_id=Ins.instructor_id where Stu.city='New York'
```

The 'Data Output' tab shows the results of the query:

	student_name	course_title	instructors_name
1	Emma Anderson	Big Data Fundamentals - Beginner	Riley White
2	Emma Anderson	ETL with Airflow - Beginner	Lee White
3	Robin Brown	Angular Deep Dive - Beginner	Riley White
4	Avery Brown	React for Beginners - Intermediate	Riley White
5	Casey Lewis	Introduction to Databases - Beginner	Robin Garcia

9. Find the name and city of all students who are enrolled in any course with the category of 'Cloud'

Query :---select distinct Stu.name as Students_Name ,Stu.city as Students_City from Students as Stu inner join Enrollments as E on Stu.student_id=E.student_id inner join Courses as C on E.course_id=C.course_id where category='Cloud'



```
85 -----9-----
86 select distinct Stu.name as Students_Name ,Stu.city as Students_City from Students as Stu
87 inner join Enrollments as E on Stu.student_id=E.student_id
88 inner join Courses as C on E.course_id=C.course_id where category='Cloud'
89
90 -----10-----
91 select title,price from Courses
92 where price >(select avg(price)from Courses)
93
94 -----11-----
95 select Ins.name,Ins.expertise from Instructors as Ins inner join Courses as C on Ins.instruc
96 group by Ins.instructor_id,Ins.name,Ins.expertise order by count(C.course_id) desc
97
98 -----12-----
99 select name from Students
```

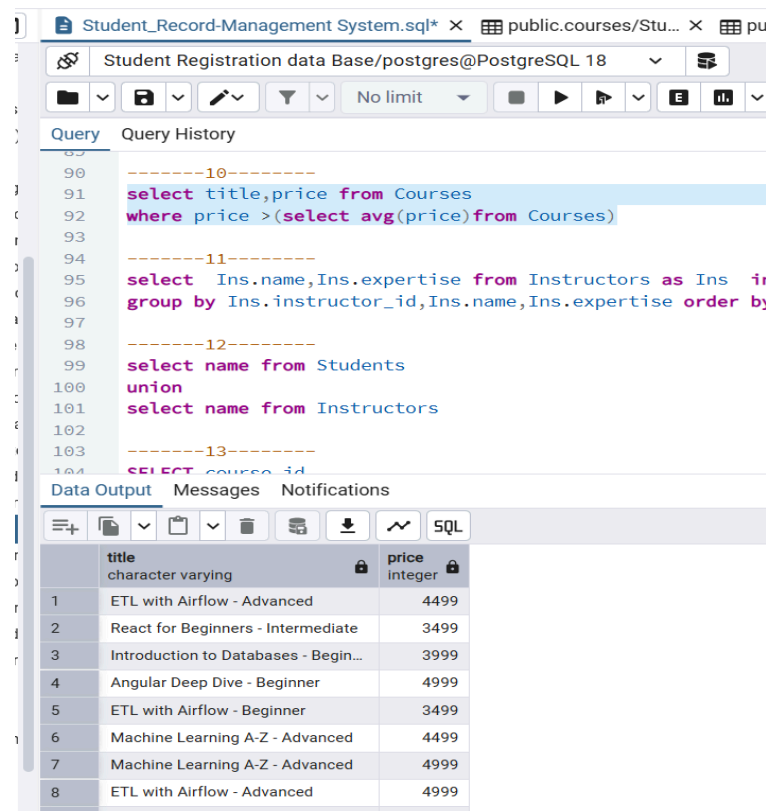
	students_name character varying	students_city character varying
1	Alex Clark	Melbourne
2	Alex Lee	Paris
3	Alex Martinez	Sydney
4	Alex Rodriguez	Singapore
5	Ava Anderson	Dubai
6	Ava Anderson	Rome
7	Ava Brown	Bengaluru
8	Ava Brown	Pune

Showing rows: 1 to 8

✓ Success

10. Retrieve the title and price of all courses that are more expensive than the average price of all courses in the entire catalog.

Query:---select title,price from Courses where price >(select avg(price)from Courses)



```
90 -----10-----
91 select title,price from Courses
92 where price >(select avg(price)from Courses)
93
94 -----11-----
95 select Ins.name,Ins.expertise from Instructors as Ins ir
96 group by Ins.instructor_id,Ins.name,Ins.expertise order by
97
98 -----12-----
99 select name from Students
100 union
101 select name from Instructors
102
103 -----13-----
104 select course_id
```

	title character varying	price integer
1	ETL with Airflow - Advanced	4499
2	React for Beginners - Intermediate	3499
3	Introduction to Databases - Begin...	3999
4	Angular Deep Dive - Beginner	4999
5	ETL with Airflow - Beginner	3499
6	Machine Learning A-Z - Advanced	4499
7	Machine Learning A-Z - Advanced	4999
8	ETL with Airflow - Advanced	4999

11. Identify the name and expertise of the instructor who teaches the highest number of courses.

Query:-- select Ins.name,Ins.expertise from Instructors as Ins inner join Courses as C on Ins.instructor_id=C.instructor_id group by Ins.instructor_id,Ins.name,Ins.expertise order by count(C.course_id) desc

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
select Ins.name,Ins.expertise from Instructors as Ins inner join Courses as C on Ins.instructor_id=C.instructor_id group by Ins.instructor_id,Ins.name,Ins.expertise order by count(C.course_id) desc
```

The query is executed, and the results are displayed in a table with two columns: name and expertise. The results are as follows:

name	expertise
Riley White	Software Engineeri...
Ava Taylor	Cloud Computing
Robin Garcia	Cloud Computing
Lee White	Software Engineeri...
Sophia Garcia	Machine Learning
Liam Martin	Machine Learning
Lee Johnson	Web Development
Alex Lee	AI

A green status bar at the bottom indicates: "Successfully run. Total query runtime: 0.00s".

12. Find all unique names that appear in either the students table or the instructors table.

Query:--- select name from Students union select name from Instructors

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
select name from Students union select name from Instructors
```

The query is executed, and the results are displayed in a table with one column: name. The results are as follows:

name
Drew Hall
Alex Clark
Robin Martin
Dana Thompson
Jamie Hall
Kris Thomas
Shawn White
Kelly Lewis

13. Find the course_id values that have at least one enrollment recorded in both the year 2024 and the year 2025.

Query:--- SELECT course_id FROM Enrollments WHERE EXTRACT(YEAR FROM enrollment_date) IN (2024, 2025) GROUP BY course_id HAVING COUNT(DISTINCT EXTRACT(YEAR FROM enrollment_date)) = 2

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the database is 'Student Registration data Base/postgres@PostgreSQL 18'. The query history shows the following SQL query:

```
-----13-----
SELECT course_id
FROM Enrollments
WHERE EXTRACT(YEAR FROM enrollment_date) IN (2024, 2025)
GROUP BY course_id
HAVING COUNT(DISTINCT EXTRACT(YEAR FROM enrollment_date)) = 2

-----14-----
select distinct course_id from Enrollments where extract(year from en
except
select distinct course_id from Enrollments where extract(year from enr

-----15-----
select title,category,price,rank() over(partition by category order by
```

The output pane shows the results of the query, displaying a list of course IDs (1 through 8) under the column header 'course_id'.

14. Find the course_id values that have enrollments in the year 2025 but not in 2024.

Query:-- select distinct course_id from Enrollments where extract(year from enrollment_date) =2025 except select distinct course_id from Enrollments where extract(year from enrollment_date)=2024

The screenshot shows a PostgreSQL query editor interface. The top bar indicates the database is 'Student Registration data Base/postgres@PostgreSQL 18'. The query history shows the following SQL query:

```
-----14-----
select distinct course_id from Enrollments where extract(year from enrollment_date) =2025
except
select distinct course_id from Enrollments where extract(year from enrollment_date)=2024

-----15-----
select title,category,price,rank() over(partition by category order by price desc)as Course

-----16-----
select title,category,price ,avg(price) over(partition by category)ascategory_avg_price,
price-avg(price) over(partition by category) as Price-Difference from Courses
```

The output pane shows the results of the query, displaying a list of course IDs (1 through 8) under the column header 'course_id'.

15. For each course category, assign a rank to each course based on its price (highest price gets rank 1). The ranking should reset for each category. Display the course title, category, price, and the calculated rank.

Query:--- select title,category,price,rank() over(partition by category order by price desc) as Course_rank from Courses;

The screenshot shows a PostgreSQL query editor with the following query executed:

```

114 -----15-----
115
116 select title,category,price,rank()
117 over(partition by category order by price desc)
118 as Course_rank from Courses;
119
120 -----16-----
121 select title,category,price ,avg(price) over(partition by category)
122 price-avg(price) over(partition by category) as Price_Difference fr
123
124 -----PART 3 :--> DATABASE MANAGEMENT AND SE
125
126 -----17-----
127 create or replace function get_course_enrollment_count(p_course_id
128

```

The Data Output tab shows the following results:

	title	category	price	course_rank
1	React for Beginners - Intermediate	Cloud	3499	1
2	Cybersecurity 101 - Intermediate	Cloud	1499	2
3	React for Beginners - Beginner	Cloud	999	3
4	Angular Deep Dive - Advanced	Cloud	999	3
5	Angular Deep Dive - Beginner	Data Science	4999	1
6	Data Visualization - Intermediate	Data Science	4999	1
7	Machine Learning A-Z - Advanced	Data Science	4499	3
8	ETL with Airflow - Advanced	Data Science	4499	3

Total rows: 40 Query complete 00:00:00.144

16. Calculate the difference between each course's price and the average price of all courses within its own category. Display the course title, price, category average price, and the calculated difference.

Query:--- select title,category,price ,avg(price) over(partition by category)as category_avg_price,price-avg(price) over(partition by category) as Price_Difference from Courses

The screenshot shows a PostgreSQL query editor with the following query executed:

```

118 as Course_rank from Courses;
119
120 -----16-----
121 select title,category,price ,avg(price)
122 over(partition by category)as category_avg_price,
123 price-avg(price) over(partition by category)
124 as Price_Difference from Courses
125
126 -----PART 3 :--> DATABASE MANAGEMENT AND SECURITY QUESTIONS-----
127
128 -----17-----
129
130 create or replace function get_course_enrollment_count(p_course_id int)
131 returns int as $$
132 declare enrollment_count int;
133

```

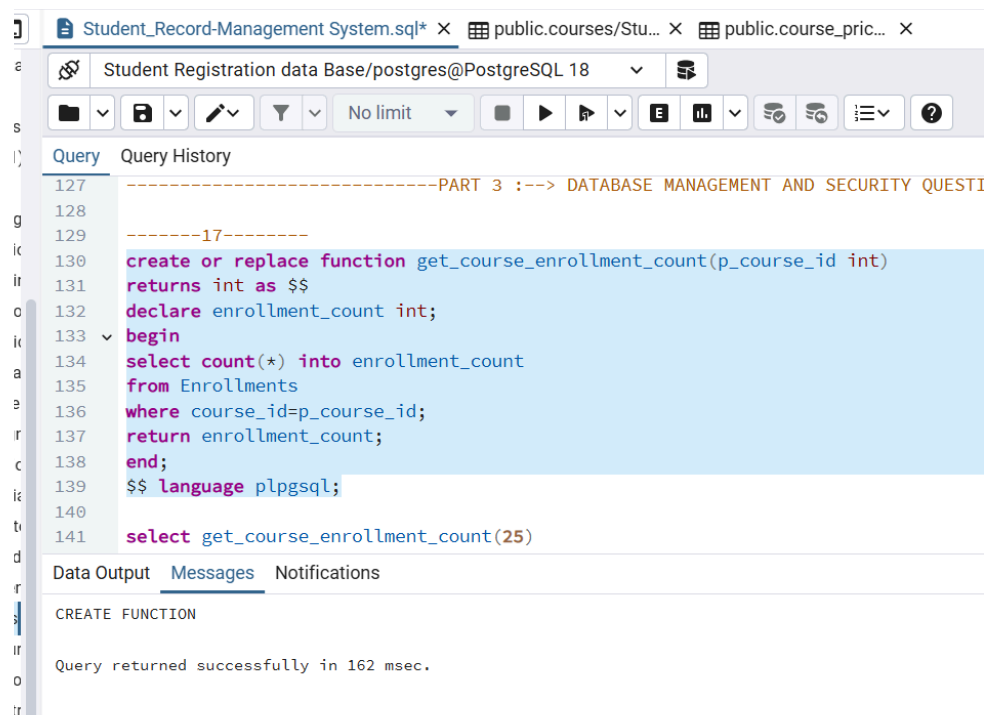
The Data Output tab shows the following results:

	title	category	price	category_avg_price	price_difference
1	React for Beginners - Intermediate	Cloud	3499	1749.0000000000000000	1750.0000000000000000
2	Angular Deep Dive - Advanced	Cloud	999	1749.0000000000000000	-750.0000000000000000
3	React for Beginners - Beginner	Cloud	999	1749.0000000000000000	-750.0000000000000000
4	Cybersecurity 101 - Intermediate	Cloud	1499	1749.0000000000000000	-250.0000000000000000
5	Introduction to Databases - Beginner	Data Science	3999	2949.0000000000000000	1050.0000000000000000
6	Angular Deep Dive - Beginner	Data Science	4999	2949.0000000000000000	2050.0000000000000000
7	Angular Deep Dive - Advanced	Data Science	999	2949.0000000000000000	-1950.0000000000000000
8	SQL Basics - Beginner	Data Science	1499	2949.0000000000000000	-1450.0000000000000000

17. Write a PostgreSQL function called `get_course_enrollment_count` that accepts a `course_id` as an integer input and returns the total number of enrollments for that specific course as an integer output.

```
Query:---create or replace function get_course_enrollment_count(p_course_id int)
returns int as $$
declare enrollment_count int;
Begin
select count(*) into enrollment_count
from Enrollments
where course_id=p_course_id;
return enrollment_count;
End;
$$ language plpgsql;

select get_course_enrollment_count(25)
```



The screenshot shows a PostgreSQL query editor window titled "Student Record-Management System.sql". The query is as follows:

```
-----PART 3 :--> DATABASE MANAGEMENT AND SECURITY QUESTI
-----17-----
create or replace function get_course_enrollment_count(p_course_id int)
returns int as $$
declare enrollment_count int;
begin
select count(*) into enrollment_count
from Enrollments
where course_id=p_course_id;
return enrollment_count;
end;
$$ language plpgsql;

select get_course_enrollment_count(25)
```

The query is executed successfully, and the output shows "CREATE FUNCTION" and "Query returned successfully in 162 msec."

18. Write the DDL to create an audit table named `course_price_audit`.

```
Query:---create table course_price_audit(audit_id serial primary key,course_id int,
old_price int,new_price int)
```

Student_Record-Management System.sql* X

Student Registration data Base/postgres@Pr

Query Query History

```
142  
143 -----18-----  
144 create table course_price_audit(  
145 audit_id serial primary key,  
146 course_id int,  
147 old_price int,  
148 new_price int  
149 )  
150  
151 -----19-----  
152 ---Creating a function for price  
153 create or replace function course  
154 returns trigger as $$  
155 begin  
156 insert into course_price_audit(  
157
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 115 msec.

19. Create a trigger that automatically logs the course_id, old_price, and new_price into the course_price_audit table whenever the price of a course is updated in the courses table.

Query:---create trigger Update_Course_PriceData
after update of price on Courses
for each row
execute function course_price_Update()
update Courses
set price =35000 where course_id=5

Student_Record-Management System.sql* X public.courses/Stu

Student Registration data Base/postgres@PostgreSQL 18

Query Query History

```
151 -----19-----  
152 ---Creating a function for price update  
153 create or replace function course_price_Update()  
154 returns trigger as $$  
155 begin  
156 insert into course_price_audit(  
157 course_id,  
158 old_price,  
159 new_price  
160 )  
161 values(  
162 old.course_id,  
163 old.price,  
164 new.price  
165 );
```

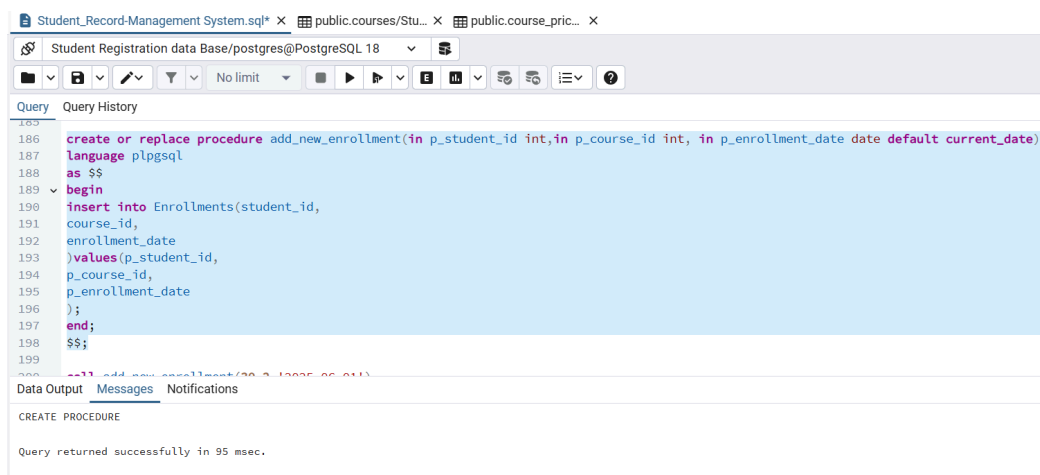
Data Output Messages Notifications

CREATE FUNCTION

Query returned successfully in 102 msec.

20. Create a stored procedure (or a function with LANGUAGE plpgsql) called add_new_enrollment that inserts a new record into the enrollments table using three parameters: p_student_id (INT), p_course_id (INT), and an optional p_enrollment_date (DATE). If no date is provided, the procedure must use the current date.

```
Query:--create or replace procedure add_new_enrollment(in p_student_id int,in
p_course_id int, in p_enrollment_date date default current_date)
language plpgsql
as $$
Begin
insert into Enrollments(student_id,
Course_id,
Enrollment_date
)values(p_student_id,p_course_id,p_enrollment_date);
End;
$$
call add_new_enrollment(30,2,'2025-06-01')
```



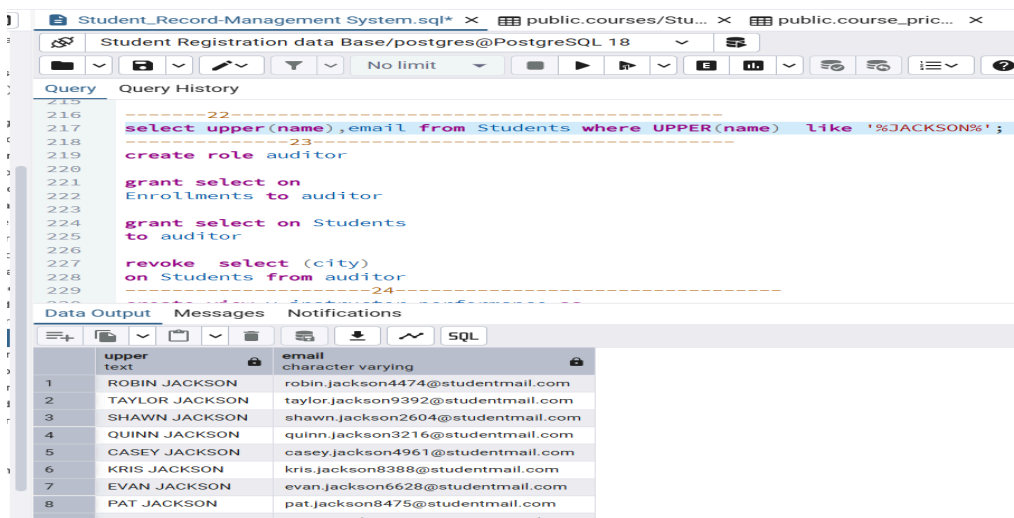
The screenshot shows a PostgreSQL query editor with the following SQL code:

```
186 create or replace procedure add_new_enrollment(in p_student_id int,in p_course_id int, in p_enrollment_date date default current_date)
187 language plpgsql
188 as $$
189 begin
190 insert into Enrollments(student_id,
191 course_id,
192 enrollment_date
193 )values(p_student_id,
194 p_course_id,
195 p_enrollment_date
196 );
197 end;
198 $$
199
200 call add_new_enrollment(30,2,'2025-06-01');
```

The query history shows the execution of the stored procedure, and the data output tab displays the message: "CREATE PROCEDURE" and "Query returned successfully in 95 msec."

21. Find the name of all students whose name contains 'Jackson' (case-insensitive). Display their name in uppercase and their email address.

```
Query:--select upper(name),email from Students where UPPER(name) like
'%JACKSON%';
```



The screenshot shows a PostgreSQL query editor with the following SQL code:

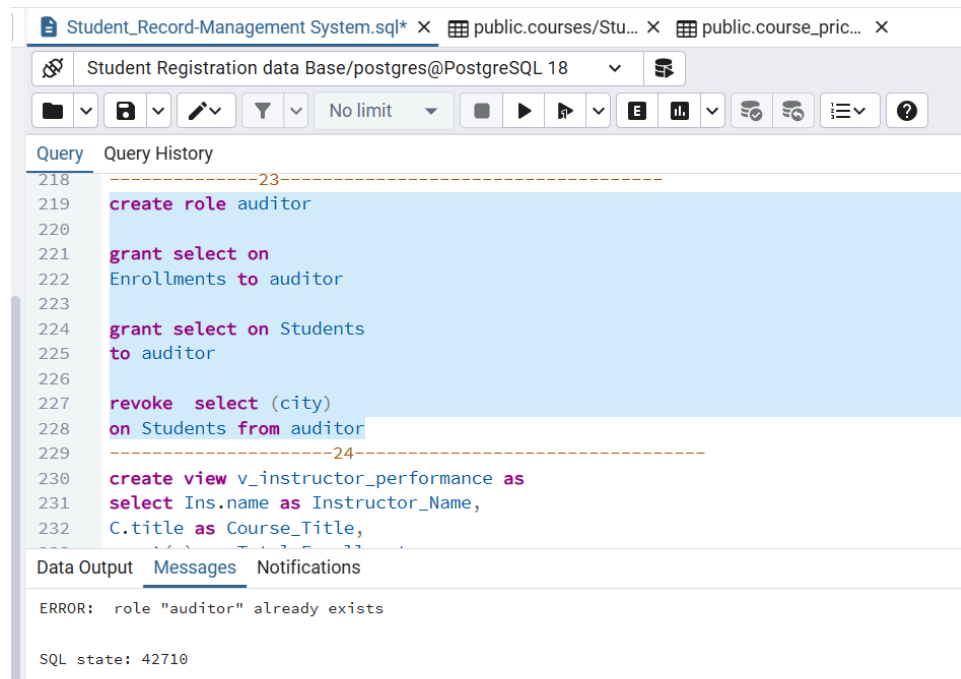
```
216 -----22-----
217 select upper(name),email from Students where UPPER(name) like '%JACKSON%';
218 -----23-----
219 create role auditor
220
221 grant select on
222 Enrollments to auditor
223
224 grant select on Students
225 to auditor
226
227 revoke select (city)
228 on Students from auditor
229 -----24-----
```

The query history shows the execution of the query, and the data output tab displays the following results:

upper text	email character varying
ROBIN JACKSON	robin.jackson4474@studentmail.com
TAYLOR JACKSON	taylor.jackson9392@studentmail.com
SHAWN JACKSON	shawn.jackson2604@studentmail.com
QUINN JACKSON	quinn.jackson3216@studentmail.com
CASEY JACKSON	casey.jackson4961@studentmail.com
KRIS JACKSON	kris.jackson8388@studentmail.com
EVAN JACKSON	evan.jackson6628@studentmail.com
PAT JACKSON	pat.jackson8475@studentmail.com

22. Create a new PostgreSQL user role named auditor. Grant this role SELECT access on the enrollments table. Revoke the SELECT privilege on the city column of the students table from the auditor role.

Query:---create role auditor
grant select on
Enrollments to auditor
grant select on Students
to auditor
revoke select (city)
on Students from auditor



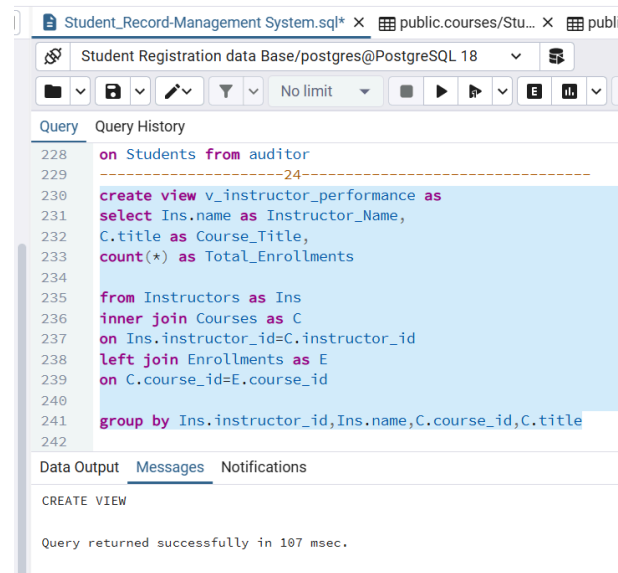
The screenshot shows a PostgreSQL query editor with the following SQL code:

```
218 -----23-----  
219 create role auditor  
220  
221 grant select on  
222 Enrollments to auditor  
223  
224 grant select on Students  
225 to auditor  
226  
227 revoke select (city)  
228 on Students from auditor  
229 -----24-----  
230 create view v_instructor_performance as  
231 select Ins.name as Instructor_Name,  
232 C.title as Course_Title,
```

The editor shows an error message: "ERROR: role "auditor" already exists". The SQL state is 42710.

23. Create a view named v_instructor_performance that shows the instructor name, the title of the courses they teach, and the total number of enrollments for each of those courses.

Query:---



The screenshot shows a PostgreSQL query editor with the following SQL code:

```
228 on Students from auditor  
229 -----24-----  
230 create view v_instructor_performance as  
231 select Ins.name as Instructor_Name,  
232 C.title as Course_Title,  
233 count(*) as Total_Enrollments  
234  
235 from Instructors as Ins  
236 inner join Courses as C  
237 on Ins.instructor_id=C.instructor_id  
238 left join Enrollments as E  
239 on C.course_id=E.course_id  
240  
241 group by Ins.instructor_id,Ins.name,C.course_id,C.title  
242
```

The editor shows a successful message: "CREATE VIEW". The query returned successfully in 107 msec.

